

Biney’s Procedural Compression: Lossless Compression as Amortized Program Search over an Archive

Biney

September 2026

Abstract

Standard lossless compressors encode a file as a transformed, shorter bitstring derived from statistical regularities in that file. This paper considers a different formulation: representing a file not as compressed *data*, but as a *procedural description* — a generating algorithm together with a seed or parameter set — such that executing the algorithm on the seed reproduces the file exactly. Under this view, compression becomes a search problem: find the smallest procedural description, within a bounded computational budget, that reconstructs the target data losslessly. This idea is a direct, resource-bounded relaxation of Kolmogorov complexity. We extend the single-file formulation to the archive setting, where many related files may share one or more procedural generators, and only a small per-file reconstruction record (an algorithm identifier, seed/parameters, and residual correction data) is stored per file. We give a formal joint optimization objective for this archive-level problem, a constrained variant with explicit search-time, reconstruction-time, and compute-budget limits, and a penalty-based relaxation suited to heuristic, gradient-free, or AI-guided search. We situate the proposal relative to Kolmogorov complexity, the minimum description length principle, compression-based similarity measures, program synthesis, and recent prediction-compression equivalence results, and we discuss computational feasibility, limitations, and an archival storage application involving decentralized, erasure-coded distribution of procedural archives. This is a theoretical position paper: it consolidates and formalizes the framework, with no implementation or empirical results yet reported.

1 Introduction

Lossless data compression algorithms in wide use today — LZ-family compressors, PPM, BWT-based schemes, and context-mixing compressors — all operate by modeling *statistical redundancy* in the input and encoding it with fewer bits than a naive representation would require. They are extremely effective and fast, and they dominate everyday use cases: network transport, filesystems, and general-purpose archiving.

This paper considers an orthogonal strategy, referred to here as **Biney’s Procedural Compression**. Instead of asking “how can this file’s bytes be re-encoded more compactly,” it asks “what is the smallest *program* that, when executed, outputs this file exactly?” A procedural description of a file consists of:

- An **algorithm** (a generating procedure), and
- A **seed** or **set of parameters** for that algorithm.

If executing the algorithm on the seed reproduces the original file byte-for-byte, and the combined size of the algorithm and seed is substantially smaller than the file itself, the pair serves as a valid compressed representation. Compression is thereby reframed as *search* over a space of candidate programs, rather than *encoding* of a fixed statistical model.

This idea is not new in the abstract — it is a direct engineering relaxation of Kolmogorov complexity, discussed further in Section 2 — but the goal of this note is not to claim theoretical novelty. It is to (a) state the framework precisely, (b) extend it from single files to whole archives of related files, where the

cost of a shared generating algorithm can be amortized across many files, and (c) lay out the resulting optimization objective, its constrained and penalty-based forms, and the practical and computational issues that any attempt to realize this framework must confront.

The motivating use case is **archival-scale, long-term storage** — specifically, long-term archival of a system referred to here as the *BLUE system* — where the objective is to trade search time and computational budget for a higher compression ratio than existing general-purpose compressors achieve. This is explicitly **not** proposed as a replacement for fast, everyday compression. It targets archives, data centers, relational databases, and other cold or long-term storage where compression is performed rarely (or once) and decompression is comparatively infrequent, so that a large, bounded, one-time search cost can be justified by a permanent reduction in stored bytes. A secondary motivation is that sufficiently compact procedural representations could, in principle, be distributed across a peer-to-peer network as a decentralized archive, combined with erasure coding or threshold reconstruction schemes so that only a subset of distributed fragments is required for full recovery.

The remainder of the paper is organized as follows. Section 2 relates the proposal to existing theory and practice. Section 3 formalizes the single-file procedural search problem. Section 4 extends this to a joint, archive-level optimization over many files sharing procedural algorithms. Section 5 gives a constrained and a penalty-based (soft-constraint) formulation suitable for heuristic or learned search. Section 6 discusses the computational feasibility of searching a combinatorially large program space under a bounded time budget. Section 7 discusses the archival storage application in more detail. Section 8 discusses limitations, and Section 9 outlines future work.

2 Related Work

Kolmogorov complexity and algorithmic information theory. The theoretical foundation of procedural compression is Kolmogorov complexity, independently introduced by Solomonoff, Kolmogorov, and Chaitin in the early-to-mid 1960s as the length of the shortest program that outputs a given string on a fixed universal machine [1, 2, 3, 5]. Kolmogorov complexity is provably **not computable** in general: there is no algorithm that, given an arbitrary string, always outputs its exact shortest generating program. This is precisely why Biney’s Procedural Compression is framed as a bounded, practical *approximation* — searching for a sufficiently small program within an explicit budget on program size, search time, and reconstruction time — rather than an attempt to compute the true minimum.

Minimum Description Length (MDL). Rissanen’s minimum description length principle [4] is the classical statistical relaxation of the same idea: rather than an uncomputable shortest program, MDL restricts the search to a class of probabilistic models and selects the model (plus the data encoded under it) with the shortest total description. MDL is the intellectual bridge between Kolmogorov complexity and practical statistical compression, and the archive-level formulation in Section 4 — where a shared “model” (procedural algorithm) is amortized across many data instances, each paying only for its own residual — is structurally analogous to MDL’s split between a model cost and a per-instance data cost, generalized here to explicit generating programs rather than probability distributions.

Compression-based similarity and shared structure. Cilibrasi and Vitányi’s normalized compression distance (NCD) [6] uses off-the-shelf compressors to define a universal similarity measure between objects, based on how much smaller a compressor can encode two objects concatenated versus separately — under the principle that similar objects share exploitable structure, which is exactly the premise underlying the multi-file, shared-algorithm formulation of Section 4 (grouping files so that a single procedural algorithm can amortize its cost across a family of “similar” files). Shared-dictionary compression schemes used in practice (e.g., dictionary-based modes of general-purpose compressors, delta and reference-based compression for related files) are a simpler, non-procedural precedent for the same amortization idea: one shared model, many cheap per-file deltas.

Program synthesis. The search step required by procedural compression — finding a program that satisfies a correctness specification, here exact byte-for-byte reproduction of a target file — is an instance of program synthesis. Syntax-Guided Synthesis (SyGuS), as formalized by Alur et al. [7], frames synthesis as finding a program within a syntactically restricted space that satisfies a semantic (here: reconstruction) specification, and is one of several candidate search paradigms — alongside genetic programming, reinforcement learning, and other heuristic or gradient-free methods — that could instantiate the abstract “search for a procedural description” step of this framework.

Context-mixing and prediction-based compressors. State-of-the-art general-purpose lossless compressors such as the PAQ family and its descendants (including cmix) achieve very high compression ratios by combining a large number of statistical predictors via a mixing model (originally weighted averaging, later a neural network) and encoding the result with an arithmetic coder [8] — trading substantial compute and memory for ratio, in a manner philosophically close to this proposal’s willingness to trade search time for ratio, but remaining within the *predictive/statistical* paradigm rather than the *generative-program* paradigm.

Prediction-compression equivalence and learned compressors. Recent work has revisited the classical equivalence between lossless compression and sequential prediction, showing that large pretrained sequence models can be used directly as strong general-purpose compressors, in some cases outperforming domain-specific compressors on out-of-domain data [9]. This line of work motivates one of the future directions noted in Section 9: using a trained model not as the compressor itself, but as a heuristic to *guide* the search over the procedural program space.

Positioning. Biney’s Procedural Compression differs from all of the above in combining three elements simultaneously: (i) an explicit, arbitrary generating *program* rather than a restricted statistical model class, (ii) a joint, archive-wide optimization objective that amortizes shared procedural algorithms across many files rather than optimizing each file independently, and (iii) an explicit, tunable trade-off between compression ratio and a bounded search/compute budget, targeted specifically at archival rather than everyday compression.

3 Problem Formulation: Single-File Procedural Search

Given a file F , a procedural description is a pair (A, S) , where A is an algorithm and S is a seed or set of parameters, such that

$$\text{Decode}(A, S) = F.$$

The compressor’s task is to search for the pair minimizing total encoded size,

$$\min_{A, S} (|A| + |S|) \quad \text{subject to} \quad \text{Decode}(A, S) = F,$$

where $|\cdot|$ denotes size in bytes. If $|A| + |S| \ll |F|$, the pair (A, S) is stored in place of F as its compressed representation.

Because exact minimization is equivalent to computing (an upper bound on) the Kolmogorov complexity of F , which is not computable in general, the search is instead conducted under explicit resource constraints:

- **Program size bound:** $|A| + |S| \leq K$ bytes.
- **Reconstruction time bound:** decoding must complete within time T .
- **Search/compute budget:** the search itself is allotted at most time or compute C .

Within these bounds, the compressor returns the best procedural representation it can find; it need not, and in general will not, find the global minimum.

4 Archive-Level Formalization: Shared Procedural Models

Compressing every file independently discards a source of redundancy that is often significant in practice: related files (e.g., files sharing a format, generator, template, or origin) may be well-explained by the *same* underlying generating algorithm, differing only in their seed, parameters, or a small residual correction. Treating each file’s procedural algorithm as a private cost, rather than a shared one, wastes this redundancy.

4.1 Notation

Symbol	Meaning
Q	Complete compressed archive.
N	Total number of files in the archive.
M	Total number of distinct shared procedural algorithms, typically $M \ll N$.
$A = \{A_1, \dots, A_M\}$	The set of shared procedural algorithms.
A_j	The j -th shared procedural algorithm.
$R = \sum_{i=1}^N R_i + G$	All reconstruction information stored in the archive.
R_i	Reconstruction information for file i .
I_i	Identifier of the shared algorithm assigned to file i .
P_i	Procedural parameters (seed) required by algorithm A_{I_i} for file i .
D_i	Residual or correction data required for exact lossless reconstruction.
G	Archive-wide information stored once (headers, tables, checksums, metadata).
F_i	The original, uncompressed file i .

4.2 Per-file reconstruction record

Each file’s stored reconstruction information decomposes as

$$R_i = (I_i, P_i, D_i), \quad |R_i| = |I_i| + |P_i| + |D_i|,$$

and the decoder must satisfy, for every file,

$$\text{Decode}(A_{I_i}, R_i) = F_i.$$

The inclusion of an explicit residual/correction term D_i is what allows a single shared algorithm to be reused even when it does not reproduce a file *exactly* on its own: any gap between the algorithm’s output and the true file is absorbed into D_i , and the reconstruction constraint above guarantees losslessness regardless of how good or poor the underlying generator is for that particular file. This is what makes joint optimization safe — sharing is never at the expense of correctness, only of size.

4.3 Before: independent per-file optimization

Compressing files independently requires

$$\min_{A_i, S_i} (|A_i| + |S_i|) \quad \text{for each file } i,$$

with total archive size

$$|Q| = \sum_{i=1}^N (|A_i| + |S_i|).$$

Every file pays for its own complete generating algorithm, even if many files would be well explained by the same generator.

4.4 After: joint archive-level optimization

Instead, the compressor searches jointly for a (generally small) collection of shared algorithms and a complete reconstruction record for every file:

$$\min_{\{A_j\}, \{R_i\}, \{I_i\}} \left(\sum_{j=1}^M |A_j| + \sum_{i=1}^N |R_i| + |G| \right),$$

equivalently, substituting the decomposition of R_i ,

$$\min_{\{A_j\}, \{R_i\}, \{I_i\}} \left(\sum_{j=1}^M |A_j| + \sum_{i=1}^N (|I_i| + |P_i| + |D_i|) + |G| \right),$$

subject to the lossless reconstruction constraint

$$\text{Decode}(A_{I_i}, R_i) = F_i \quad \forall i \in \{1, \dots, N\}.$$

Because G is stored exactly once for the entire archive rather than once per file, its cost is amortized over N and becomes negligible for large archives.

This objective jointly determines:

- how many shared algorithms M should exist,
- which files should be assigned to (share) each algorithm,
- when instantiating a new algorithm is preferable to reusing an existing one at the cost of a larger residual D_i , and
- the trade-off between the size of the shared algorithms and the aggregate size of the per-file reconstruction records.

4.5 Special cases

The formalism does not presuppose a fixed architecture; M is itself part of what is being optimized, and the two extremes are simply boundary cases of the same objective:

1. **Single universal algorithm** ($M = 1$): every file is reconstructed as $F_i = A(R_i)$.
2. **Multiple shared algorithms** ($M > 1$): each file is reconstructed as $F_i = A_{I_i}(R_i)$, with files clustered against whichever algorithm minimizes their combined algorithm-amortization and residual cost.
3. **Any intermediate arrangement** between these extremes, if it yields a smaller total archive size $|Q| = \sum_j |A_j| + \sum_i |R_i| + |G|$.

Rather than independently minimizing every file's compressed size, the compressor minimizes the storage cost of the archive **as a whole**, subject to exact reconstruction of every file.

5 Constrained and Penalty-Based Optimization

5.1 Constrained form

The simplest joint objective in Section 4 ignores computational cost. A more realistic formulation reintroduces the practical constraints from Section 3 at the archive level:

$$\begin{aligned}
& \text{Minimize} && \sum_{j=1}^M |A_j| + \sum_{i=1}^N |R_i| + |G| \\
& \text{subject to} && \text{Decode}(A_{I_i}, R_i) = F_i, \quad \forall i, \\
& && \text{Search Time} \leq T, \\
& && \text{Reconstruction Time} \leq R_{\max}, \\
& && \text{Compute Budget} \leq C, \\
& && \text{Size of Compressed Archive} \leq K.
\end{aligned}$$

5.2 Penalty-based relaxation

Hard constraints are awkward for gradient-free, heuristic, or AI-guided search procedures, which typically operate more naturally on a single scalar objective. An equivalent penalty-based formulation folds each constraint into the objective as a soft penalty:

$$\min \sum_{j=1}^M |A_j| + \sum_{i=1}^N |R_i| + |G| + \lambda_D L_{\text{decode}} + \lambda_T L_{\text{search}} + \lambda_R L_{\text{reconstruction}} + \lambda_C L_{\text{compute}} + \lambda_K L_{\text{archive}},$$

where L_{decode} penalizes incorrect reconstruction, L_{search} penalizes search time exceeding T , $L_{\text{reconstruction}}$ penalizes reconstruction time exceeding $R_{\max}$, L_{compute} penalizes compute usage exceeding C , L_{archive} penalizes total archive size exceeding K , and the λ coefficients weight the relative importance of each penalty. Rather than immediately rejecting a candidate solution that violates a constraint, the optimizer assigns it a higher objective value, naturally steering the search toward smaller archive representations that also respect the desired computational limits.

This reformulation converts a multi-constraint problem into a single scalar objective directly compatible with AI-guided search, heuristic search, evolutionary/genetic methods, clustering-based assignment of files to shared algorithms, reinforcement learning, or other gradient-free combinatorial optimization techniques — without requiring an exhaustive search of the underlying program space.

6 Computational Feasibility

The space of candidate programs of even modest length is astronomically large: for example, there are on the order of 2^{4000} possible procedural programs of length 500 bytes under an unrestricted binary representation. No realistic system can search this space exhaustively.

The framework handles this not by reducing the space, but by making the search **anytime and budget-bounded**: setting a search-time limit T (for instance, $T = 76$ hours) does not imply exploring the full space in that time; it simply means the search explores as much of the reachable subset of candidate programs as the budget allows and returns its best candidate when the budget is exhausted. Increasing T increases computational cost but also the probability of finding a smaller procedural representation. This anytime property is what makes the framework compatible with an archival setting: search cost is paid once, at compression time, and can be scaled arbitrarily against the value of the resulting storage savings, independent of how large the theoretical search space is.

7 Application: Archival-Scale and Decentralized Storage

The primary motivating application is the long-term archival of large-scale systems — data centers, relational databases, and similar long-term storage — where compression is performed comparatively rarely relative to the lifetime of the stored data, so that a large, one-time search cost is amortized over a long storage horizon. This is explicitly distinct from everyday, latency-sensitive compression use cases, which this framework does not target.

A secondary application follows from the archive-level formulation of Section 4: if Biney’s Procedural Compression discovers sufficiently compact procedural representations for large portions of an archive — or the archive as a whole — those procedural programs (shared algorithms plus per-file reconstruction records) are themselves small enough to be distributed across a peer-to-peer network as a decentralized archive. Combined with erasure coding or threshold reconstruction — where only k of n distributed fragments, as in Reed–Solomon-style maximum-distance-separable codes [10], are required to recover the complete archive — this could in principle improve the storage efficiency, resilience, and decentralization of long-term archival storage relative to full-replica or conventionally compressed distribution. Archival storage is one motivating application among several, but it is the primary one driving this line of work.

8 Discussion and Limitations

Several issues follow directly from the formulation above and should be stated explicitly, since this remains a theoretical proposal with no empirical validation:

- **Incomputability of the exact objective.** As in Section 2, the true minimum of any of the objectives above is not computable in general. Every practical instantiation of this framework is necessarily a heuristic approximation, and its quality is entirely dependent on the strength of the search procedure used, not on the objective itself.
- **Search cost versus benefit.** The framework is only worthwhile when search cost is small relative to the value of the storage savings achieved, and when reconstruction cost is acceptable for the target use case. Both trade-offs are use-case dependent and are left as explicit, tunable constraints (T , $R_{\max}$, C , K) rather than fixed defaults.
- **Residual/correction data as a safety net, not a shortcut.** The role of D_i (Section 4) is essential: it guarantees exact reconstruction even when a shared algorithm imperfectly matches a given file. However, if the search process relies too heavily on D_i rather than on a genuinely good procedural match, the archive-level objective degenerates toward ordinary per-file compression with extra overhead from algorithm bookkeeping, and the amortization benefit of sharing is lost. A concrete algorithm-assignment procedure needs to guard against this failure mode, though none is proposed here.
- **Reconstruction-time asymmetry.** Nothing in the formulation guarantees that decoding is fast; a procedural description that is very small may still be expensive to execute. The reconstruction-time constraint $R_{\max}$ is included precisely to bound this, but choosing $R_{\max}$ appropriately for a given archival use case is itself an open design question.
- **No prototype or empirical results.** This paper formalizes and consolidates the framework; it does not report an implementation, a concrete search algorithm, or measurements against existing compressors. All feasibility claims here are structural (bounded search over a large but finite space) rather than empirical.

9 Future Work

Several extensions follow naturally from the framework described above:

- **AI- or reinforcement-learning-guided search** over the procedural program space, potentially using a pretrained sequence model as a heuristic guide rather than as the compressor itself, in the spirit of recent prediction-compression equivalence results (Section 2).
- **Hybrid approaches** that combine procedural search with existing statistical/entropy-coding compressors — for example, using a procedural generator to produce a first-pass approximation and an existing compressor (e.g., a context-mixing compressor) to encode the residual D_i .
- **Domain-specific procedural generators** for images, audio, video, documents, and 3D assets, where strong structural priors (e.g., known parametric or fractal generators, layout grammars, or codecs) may make procedural search far more tractable than in the fully general binary-program setting of Section 6.
- **A concrete algorithm-assignment and clustering procedure** for the archive-level objective of Section 4, connecting it more explicitly to compression-based clustering methods such as normalized compression distance (Section 2).
- **An empirical prototype** and benchmark comparison against existing archival-grade compressors (e.g., context-mixing compressors, general-purpose dictionary compressors with shared dictionaries) on representative archival corpora, to test whether the theoretical amortization benefit of Section 4 is realizable in practice.

10 Conclusion

This paper consolidates and formalizes Biney’s Procedural Compression: a reframing of lossless compression as bounded search for a generating program, rather than as encoding of a statistical model. It extends the single-file formulation to a joint, archive-level optimization in which one or more shared procedural algorithms are amortized across many related files, each paying only for its own algorithm identifier, seed/parameters, and residual correction data, under an explicit lossless reconstruction guarantee. Constrained and penalty-based variants of the objective are given to support practical, resource-bounded, and potentially AI- or heuristic-guided search. The framework is positioned relative to Kolmogorov complexity, the minimum description length principle, compression-based similarity, program synthesis, and recent prediction-compression equivalence work, and its primary motivating application — long-term, archival-scale, and potentially decentralized storage — is discussed alongside its central open problems: incomputability of the exact objective, search-cost/benefit trade-offs, and the absence, to date, of any concrete search algorithm or empirical evaluation. These remain the central open problems for future work.

References

- [1] R. J. Solomonoff, “A Formal Theory of Inductive Inference, Parts I and II,” *Information and Control*, vol. 7, no. 1–2, pp. 1–22, 224–254, 1964.
- [2] A. N. Kolmogorov, “Three Approaches to the Quantitative Definition of Information,” *Problems of Information Transmission*, vol. 1, no. 1, pp. 1–7, 1965.
- [3] G. J. Chaitin, “On the Length of Programs for Computing Finite Binary Sequences,” *Journal of the ACM*, vol. 13, no. 4, pp. 547–569, 1966.

- [4] J. Rissanen, “Modeling by Shortest Data Description,” *Automatica*, vol. 14, no. 5, pp. 465–471, 1978.
- [5] M. Li and P. Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications*, 3rd ed. Springer, 2008.
- [6] R. Cilibrasi and P. M. B. Vitányi, “Clustering by Compression,” *IEEE Transactions on Information Theory*, vol. 51, no. 4, pp. 1523–1545, 2005.
- [7] R. Alur, R. Bodík, G. Juniwal, M. M. K. Martin, M. Raghothaman, S. A. Seshia, R. Singh, A. Solar-Lezama, E. Torlak, and A. Udupa, “Syntax-Guided Synthesis,” in *Proc. IEEE Int. Conf. Formal Methods in Computer-Aided Design (FMCAD)*, 2013, pp. 1–17.
- [8] M. V. Mahoney, “Adaptive Weighing of Context Models for Lossless Data Compression,” Florida Institute of Technology, Tech. Rep. CS-2005-16, 2005.
- [9] G. Delétang, A. Ruoss, P.-A. Duquenne, E. Catt, T. Genewein, C. Mattern, J. Grau-Moya, L. K. Wenliang, M. Aitchison, L. Orseau, M. Hutter, and J. Veness, “Language Modeling Is Compression,” *arXiv:2309.10668*; *Int. Conf. on Learning Representations (ICLR)*, 2024.
- [10] I. S. Reed and G. Solomon, “Polynomial Codes Over Certain Finite Fields,” *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960.

This paper consolidates and formalizes three earlier working notes: “Proposal: Biney’s Procedural Compression,” “#6 Optimization for Biney’s Procedural Compression,” and “Clarification” (notation note), all dated July 2026.